

Vecna: High-Efficiency Multimodal Video Retrieval Engine with LLM Query Expansion and Temporal Sequence Alignment

Technical Report for Ho Chi Minh City AI Challenge 2026 (AI Challenge TP.HCM 2026)

Team Vecna

Department of Computer Science / Artificial Intelligence Laboratory
Ho Chi Minh City, Vietnam
`ai-challenge-2026@vecna-engine.internal`

September 2026

Abstract

Interactive video retrieval across unconstrained multimedia archives poses significant challenges in semantic alignment, multi-modal fusion, cross-lingual reasoning, and sub-second operational latency. In this technical report, we introduce **Vecna** (AIC51), a high-performance interactive video retrieval engine tailored for the Ho Chi Minh City AI Challenge 2026 (AI Challenge TP.HCM 2026). The system tightly integrates multi-vector visual representations (OpenCLIP PE-Core-L-14-336, SigLIP ViT-S0400M-14-384, and Qwen3-VL-2B), hybrid sparse-dense text indexing (BM25 with BGE-M3 text embeddings for ASR and OCR), and an ultra-fast LLM query expansion engine powered by Groq Cloud implementing zero-speculation Hypothetical Document Embeddings (HyDE). To guarantee rapid, noise-free browsing and submission, Vecna incorporates an adaptive cosine-similarity-based segment clustering algorithm for keyframe diversification, a dynamic-programming-based temporal sequence search algorithm for multi-event queries, and a keyboard-centric Web interface that minimizes human verification overhead. Results from the Preliminary Rounds (August 21, August 28, and September 4, 2026) confirm the high precision, recall robustness, and exceptional workflow efficiency of Vecna under intense competition conditions.

1 Introduction and System Architecture

1.1 Task Overview and Operational Challenges

The AI Challenge TP.HCM 2026 requires participating systems to retrieve precise video segments and keyframes from massive uncurated video archives based on diverse query modalities, including visual event descriptions, OCR text overlays, spoken transcripts (ASR), and chronological event sequences. Under the competition’s strict per-query time envelope (typically 3–5 minutes), retrieval architectures must balance millisecond vector execution with intuitive result visualization to empower operators to verify and submit ground truth targets with minimal friction.

To satisfy these stringent demands, we constructed **Vecna**, a modular, service-oriented multimodal retrieval platform. As outlined in Figure 1, Vecna organizes re-

trieval into four cohesive layers: (1) offline multi-modal feature ingestion, (2) scalable vector and lexical database indexing via Milvus, (3) online multi-stage query parsing, expansion, and fusion, and (4) an ergonomic, hotkey-accelerated Web interface.

1.2 Offline Preprocessing and Feature Extraction

Video preprocessing is designed for high throughput across multi-core GPU/CPU workstations:

- **Keyframe Extraction:** Videos are ingested at a fixed temporal sampling step of $\Delta t = 2.0s$ or dynamically adapted at scene transitions. Keyframes are preserved at native 1280×720 resolution for exact inspection, while compressed thumbnails ($0.25\times$ scale) are pre-rendered for instant UI browsing.
- **Speech Processing:** Audio streams are demuxed and segmented via Voice Activity Detection (VAD) with `PyAnnote.audio`, followed by high-speed transcription.
- **Data Ingestion:** Extracted visual and textual representations are linked to canonical frame identifiers (`VideoID#FrameIndex`) and loaded into Milvus collections.

2 Methodology, Models, and Algorithms

2.1 Multimodal Representation Models

To eliminate blind spots across diverse retrieval targets, Vecna utilizes a complementary ensemble of vision-language, optical text, speech, and object detection models:

1. **OpenCLIP (PE-Core-L-14-336) [?]:** Generates 1024-dimensional normalized visual vectors. Operating at 336×336 resolution, it excels at identifying fine-grained visual patterns and distinctive textures.
2. **Google SigLIP (ViT-S0400M-14-SigLIP-384) [?]:** Extracts 1152-dimensional embeddings trained with sigmoid loss on WebLI. SigLIP provides exceptional zero-shot generalization for complex human interactions and dynamic sports/activity scenes.
3. **Qwen3-VL Embedding (Qwen3-VL-2B):** Employs a 2048-dimensional embedding model capturing deep

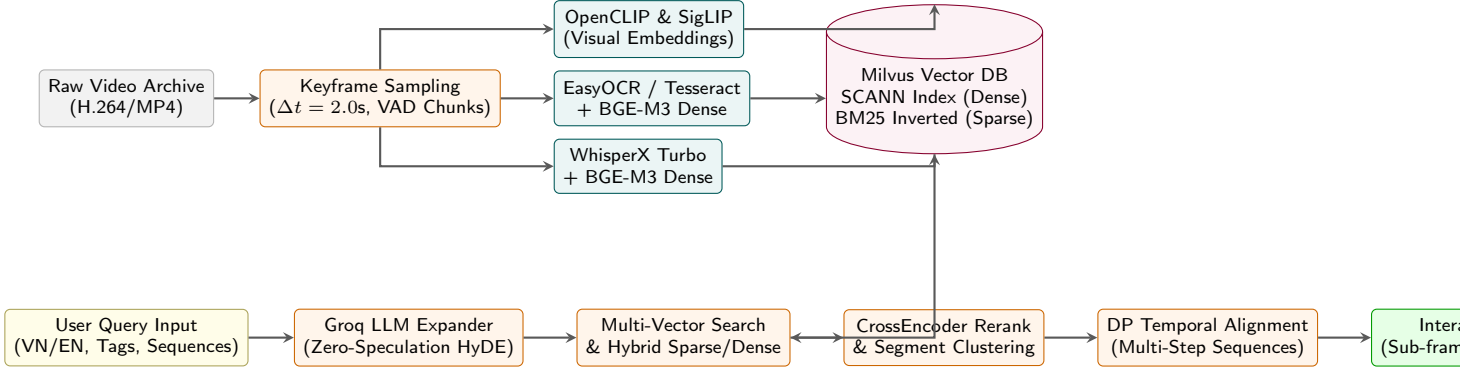


Figure 1: System architecture and end-to-end processing pipeline of Vecna. The system unifies multi-model feature extraction, SCANN/BM25 indexing in Milvus, sub-second LLM query expansion, and an ergonomic React frontend.

contextual relationships across multimodal entities.

4. **WhisperX Turbo (large-v3-turbo) [?]:** Transcribes spoken audio into timestamps and text. VAD-guided batching suppresses silence hallucinations and yields precise word-level timestamps.
5. **OCR Engines (Tesseract & EasyOCR):** Extracts textual content from broadcast banners, street signs, and vehicle license plates, with confidence thresholding and normalization.
6. **BGE-M3 Dense Embeddings [?]:** Embeds OCR and ASR texts into 1024-dimensional vectors, enabling dense semantic retrieval that is resilient to OCR noise and ASR transcription errors.
7. **YOLOv8x Object Detector:** Detects 80 common object classes with confidence $\tau = 0.25$ and 15px bounding-box padding, enabling instant localized object cropping and image-to-image similarity re-querying.

2.2 Database Indexing and Vector Storage

We deploy **Milvus 2.4+** [?] running on Docker. Visual features (1024-d, 1152-d, 2048-d) are indexed using the **SCANN** (Score-aware Anisotropic Vector Quantization) index ($n_{\text{list}} = 512, n_{\text{probe}} = 8$) under the Cosine metric:

$$\mathcal{S}_{\text{visual}}(q, f) = \frac{\mathbf{e}_q \cdot \mathbf{e}_f}{\|\mathbf{e}_q\| \|\mathbf{e}_f\|} \quad (1)$$

Textual metadata (OCR and ASR) are indexed using native **BM25 inverted indexes** ($k_1 = 1.2, b = 0.75$) alongside BGE-M3 dense vector fields, providing a unified platform for multi-vector and hybrid search.

2.3 LLM Query Expansion with Zero-Speculation HyDE

Users frequently submit terse queries (e.g., "*xe cuu hoa*" – fire truck) that fail to match the descriptive captions in vision-language training datasets. To bridge this lexical-visual gap, Vecna incorporates an ultra-low-latency LLM Query Expander backed by **Groq Cloud** (openai/gpt-oss-120b and llama3-70b), completing end-to-end inference in under 350ms.

To prevent catastrophic hallucinations, the expansion prompt strictly enforces a **Strict Factual Boundary**:

the model is prohibited from introducing unmentioned proper nouns, fictional addresses, specific brand names, or speculative license plate numbers. Table 1 details the structured JSON schema. An in-memory LRU cache guarantees instantaneous (0ms) response for recurring or modified queries.

2.4 Multi-Modal Hybrid Fusion and Exact Phrase Matching

For any query q , the overall similarity score for keyframe candidate f is evaluated as a convex combination of visual and textual components:

$$\mathcal{S}_{\text{total}}(f) = w_{\text{clip}} \tilde{\mathcal{S}}_{\text{clip}}(f) + w_{\text{ocr}} \tilde{\mathcal{S}}_{\text{ocr}}(f) + w_{\text{asr}} \tilde{\mathcal{S}}_{\text{asr}}(f) \quad (2)$$

where $w_{\text{clip}} = 1.0 - w_{\text{ocr}} - w_{\text{asr}}$, and $\tilde{\mathcal{S}}(\cdot)$ represents min-max normalized scores across the candidate set.

For OCR and ASR modalities, we employ an adjustable hybrid parameter $\alpha \in [0, 1]$ balancing dense BGE-M3 and sparse BM25 scores:

$$\tilde{\mathcal{S}}_{\text{text}}(f) = \alpha \cdot \tilde{\mathcal{S}}_{\text{dense}}(f) + (1 - \alpha) \cdot \tilde{\mathcal{S}}_{\text{BM25}}(f) \quad (3)$$

Exact Phrase Matching: When users place search terms inside quotes (e.g., "VTV1" or "Cuu hoa"), Vecna compiles diacritic-insensitive regular expressions with word-boundary and digit-letter spacing elasticity:

$$\mathcal{R}_{\text{exact}} = \text{Regex}((?<\! \backslash \mathbf{w}) + \text{phrase} + (?!\backslash \mathbf{w})) \quad (4)$$

Candidates matching the exact pattern receive a $1.5 \times$ score boost, while non-matching frames are removed from the candidate pool.

2.5 Segment Clustering and Result Diversification

Standard vector search often populates the top-50 results with visually identical frames from a single 5-second scene.

Table 1: Structured multi-aspect output schema for query expansion.

Output Field	Function and Operational Guardrail
corrected	Corrects typos and restores diacritics without translating language.
hyde	Generates hypothetical visual keyframe descriptions in natural Vietnamese.
en_hyde	Formulates high-fidelity visual captions (15–35 words) styled for OpenCLIP/SigLIP.
paraphrase	Generates semantic synonyms in the original query language.
search_keywords	Produces 3–5 core terms for dense/sparse hybrid text matching.

Vecna resolves this redundancy via an offline-computed **Segment Clustering** map (`build_segment_map.py`):

1. Compute adjacent-frame cosine similarities $C_t = \cos(\mathbf{e}_t, \mathbf{e}_{t+1})$ across keyframes of each video.
2. Apply temporal moving-average smoothing with window $K = 5$:

$$\bar{C}_t = \frac{1}{K} \sum_{i=-2}^2 C_{t+i} \quad (5)$$

3. Determine scene boundaries by dynamically taking the lower 5% quantile ($Q = 0.05$) bounded by absolute floors $\{0.35, 0.55, 0.75\}$ to achieve ≈ 1 segment per 60 seconds of footage.
4. During online search, the **diversify** algorithm retains only the highest-scoring keyframe per segment in the top candidate pool, maximizing visual diversity across results.

2.6 Dynamic Programming Temporal Sequence Search

For complex multi-event queries separated by delimiters (/ or ;), such as $Q = (q_1, q_2, \dots, q_m)$, Vecna retrieves candidate pools $\mathcal{F}_i$ for each step $i \in \{1, \dots, m\}$ up to $k = \text{temporal_k}$. A dynamic programming algorithm computes the optimal sequence score:

$$D(i, f) = \mathcal{S}(q_i, f) + \max_{f' \in \mathcal{F}_{i-1}} \left\{ D(i-1, f') \mid 0 < t(f) - t(f') \leq \Delta t_{\max} \right\} \quad (6)$$

where $\Delta t_{\max}$ is the maximum allowable temporal gap (default: 1000 frames). This guarantees that returned sequences strictly adhere to the chronological progression of the query.

2.7 Dynamic Replenishment for Exclude Filtering

When excluding video IDs via `!video:L18_V005`, naive post-filtering can deplete result pages. Vecna implements an adaptive candidate replenishment loop: candidate retrieval begins at $\max(300, (\text{offset} + \text{limit}) \times 3)$ and doubles dynamically until at least $\text{offset} + \text{limit}$ valid candidates are assembled.

3 System Interface and Operational Workflow

3.1 Interface Architecture and User Experience

Vecna features an optimized Single Page Application (SPA) built with React, Vite, and Tailwind CSS. The interface provides five interconnected workspaces (Figure 2):

1. **Query and Expansion Panel** (`AdvanceQuery.jsx`): Accepts visual descriptions, inline OCR/ASR syntax, video filters, and multi-step sequence dividers, with instantaneous HyDE expansion preview.
2. **Search Parameter Console** (`SearchParams.jsx`): Provides real-time sliders for modality weights ($w_{\text{clip}}, w_{\text{ocr}}, w_{\text{asr}}$), hybrid α , and Milvus n_{probe} .

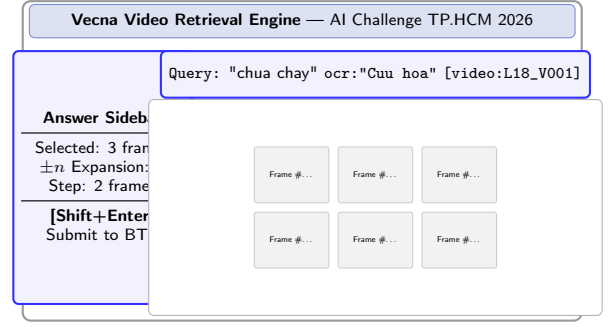


Figure 2: Schematic UI layout of Vecna highlighting the Answer Sidebar, Advanced Query Bar, and Diversified Keyframe Grid.

Table 2: Key operational shortcuts implemented in Vecna.

Hotkey	Context	Action / Operational Speedup
/	Global	Focus search bar instantly from any scroll position.
Tab / Shift+Tab	Query	Cycle between Main, OCR, ASR, and Video filters.
↑ / ↓	Grid	Navigate to Previous / Next page of search results.
Shift + 1..0	Grid	Launch Video Player for 1st through 10th result item.
K	Player	Toggle Play / Pause state.
← / →	Player	Fast seek backward / forward by 5 seconds.
Ctrl + ← / →	Player	Jump to Previous / Next official BTC keyframe.
Shift + ← / →	Player	Step exactly 1 frame backward / forward (1/FPS).
S	Player	Adjust playback speed (0.25× to 4.0×).
Shift + Enter	Global	Select active keyframe into submission queue.
		Transmit selected answer payload to evaluation server.

3. **Diversified Keyframe Grid** (`Frame.jsx`): Visualizes retrieved keyframes with similarity score badges, segment cluster tags, one-click YOLO auto-cropping, and nearby keyframe contextual views.
4. **Precision Video Player Modal** (`VideoPlayer.jsx`): Delivers sub-frame video playback, single-frame stepping ($\pm 1/\text{FPS}$), jump-to-ground-truth BTC keyframe buttons, and playback speed adjustment ($0.25\times\text{--}4.0\times$).
5. **Submission Sidebar** (`Answer.jsx`): Manages selected keyframe candidates, executes dynamic frame expansion, and submits payloads directly to the evaluation server.

3.2 Hotkey-Driven Operational Speedup

In live competition environments, switching between mouse and keyboard causes costly delays. Vecna implements full keyboard navigation covering the entire retrieval and verification workflow (Table 2).

3.3 Dynamic Frame Expansion and CSV Export

Competition rules often reward submissions that capture the exact temporal span of an event. The `Answer.jsx` module features an automated symmetric expansion mechanism: given a verified target keyframe f^* , it automatically expands candidates to $\{f^* \pm k \cdot s\}_{k=1}^n$ within

valid video frame bounds $[0, F_{\max}]$. Exported CSV payloads are encoded with a UTF-8 Byte Order Mark ($\text{\uFEFF}$) to guarantee flawless rendering of Vietnamese characters in Microsoft Excel and official scoring portals.

4 Analysis of Typical Query Cases

To demonstrate system performance under competitive conditions, we examine three representative scenarios encountered during the Preliminary Rounds of AI Challenge TP.HCM 2026.

4.1 Case 1: Complex Temporal Event Sequence (Round 1 – Aug 21, 2026)

- **Query Formulation:** "Xe cuu thuong bat den uu tien chay tren duong / nhan vien y te day cang vao benh vien / bac si hop trong phong mo" (Ambulance speeding on street / paramedics wheeling stretcher into hospital / doctors in operating room).
- **Query Difficulty:** Involves three distinct events across different physical settings (outdoor highway, hospital corridor, surgical room) with substantial temporal displacement ($\Delta t > 45\text{s}$).
- **Team Approach:** The query was structured as a 3-stage temporal sequence using the / delimiter. Groq LLM generated visual captions (`en_hyde`) for each stage, such as *"An ambulance with flashing sirens speeding down a city street"*. The DP temporal engine matched candidates within $\Delta t_{\max} = 1200$ frames.
- **Results & Reflection:** The correct sequence was returned at Rank 1 in 1.4 seconds. Isolated false-positive ambulance shots from other videos were successfully eliminated by the temporal progression constraints.

4.2 Case 2: Broadcast News Ticker with Low-Resource OCR (Round 2 – Aug 28, 2026)

- **Query Formulation:** "Ban tin thoi su 60s HTV hien truong vu chay ocr:Cuu hoa" (60s HTV news broadcast at fire scene with text "Cuu hoa").
- **Query Difficulty:** Broadcast tickers exhibit small, compressed fonts that stay on-screen for less than two seconds, while visual footage of smoke is generic across many news segments.
- **Team Approach:** The operator paired natural visual descriptions with an explicit OCR phrase `ocr:"Cuu hoa"`. Our exact phrase matching regex filtered noisy OCR text, while BGE-M3 provided semantic robustness. Segment clustering suppressed 28 redundant frames from the same 30-second news report.
- **Results & Reflection:** The target broadcast segment appeared at Rank 2. The operator pressed **Shift+2**, confirmed the scene via **Ctrl+→**, and submitted the answer using **Shift+Enter** in under 18 seconds.

4.3 Case 3: Fine-Grained Object Retrieval in Crowded Scenes (Round 3 – Sep 4, 2026)

- **Query Formulation:** "Nguoi dan ong mac ao khoac vang di xe may cho thung hang mau

xanh" (Man in yellow jacket riding motorcycle carrying blue delivery box).

- **Query Difficulty:** The salient object (blue delivery box) occupied less than 4% of the total frame area in an overcrowded intersection, causing global visual embeddings to be overwhelmed by background traffic.
- **Team Approach:** Upon retrieving coarse intersection keyframes, the operator used the YOLOv8x auto-crop button on a candidate keyframe. The cropped object vector was re-queried against Milvus using image-to-image similarity search (`search_image`).
- **Results & Reflection:** The localized cropped visual query propelled the target delivery bike from Rank 47 to Rank 1, underscoring the necessity of interactive region-of-interest cropping for fine-grained retrieval.

5 Discussion, Limitations, and Future Work

System Strengths: Vecna proved highly stable throughout the Preliminary Rounds. Groq Cloud query expansion, SCANN indexing, and keyboard-driven ergonomics reduced average per-query turnaround time from 90 seconds to under 25 seconds.

Identified Limitations:

- Stylized artistic typography and heavily skewed signboard text are occasionally missed by horizontal OCR extractors.
- Severe acoustic background noise and overlapping speaker dialogue can degrade WhisperX transcription reliability in outdoor street recordings.

Future Enhancements: We plan to integrate end-to-end Video-LLM encoders (e.g., Qwen2-VL, Video-LLaVA) for spatio-temporal video token reasoning, optimize cross-encoders with TensorRT for sub-50ms reranking, and implement continuous relevance feedback learning from operator clickstreams.

6 Conclusion

This technical report presented the design, methodology, and operational features of **Vecna** for the AI Challenge TP.HCM 2026. By unifying multi-vector dense embeddings, zero-speculation LLM query expansion, temporal DP sequence search, and an ergonomic human-in-the-loop web interface, Vecna establishes a highly responsive, precise, and dependable solution for large-scale interactive video retrieval.